

iW-ASCEJ-01 · iWave SDR Platform (ZU47DR RFSoc)

User Manual

Installation, operation and troubleshooting

Document number	iW-ASCEJ-01-User-Manual-R1.0-REL1.0
Revision	R1.0
Package release	REL1.0 (software 1.2.5)
Target device	xczu47dr-ffvg1517-2-i
Date	05-Oct-2026

Prepared by	
Reviewed by	
Approved by	

Company confidential - for project use. Printed copies are uncontrolled.

Contents

Introduction

- Purpose
- Scope
- Package layout
- Conventions
- Related documents

System overview

- Signal chains
- Default operating point (firmware boot values)
- Bandwidth facts that shape operation

Requirements

- Target hardware
- Host computer
- Firmware build

Installation

- Step 1: verify the package
- Step 2: firmware
- Step 3: QDMA driver (`iwfg`)
- Step 4: GUI and helpers
- Step 5: verify host-side (optional, no board needed)

Quick start

- First run in simulation (no hardware)
- First live capture
- First transmit loopback

Workstation GUI operation

RF data converter control (RFDC tab)

H2C CONTROL window

DAC input source and host transmit

Capture and transmit together

- Why the capture rate matters to transmit
- What the GUI does (1.2.5)
- Planning another configuration

Firmware serial console

Command-line control: `rfdc_ctl`

Troubleshooting

- Where to look first
- Symptoms and remedies
- Getting back to a known state

Known limitations

Appendix A: PCIe control mailbox (summary)

Introduction

Purpose

This manual explains how to install, operate and troubleshoot the iWave SDR platform built on the AMD/Xilinx Zynq UltraScale+ RFSoc xczu47dr-ffvg1517-2-i (project iW-ASCEJ-01, package release REL1.0, software 1.2.5). It is written for the engineers who bring up the board, run the workstation GUI and integrate the platform into a larger system.

Scope

The platform has three parts:

- **Bare-metal firmware** on the A53. It runs on the Cortex-A53 of the RFSoc, configures the clock tree (LMK04828, LMX2594), the RF data converter, the on-chip DDS and the stream routing. It serves control commands from the host through a PCIe register mailbox, and from a UART menu.
- **The host driver and helpers.** The `iwfg` QDMA kernel driver exposes `/dev/iwfg0` (card-to-host capture) and `/dev/iwfg1` (host-to-card transmit). Two user-space helpers move the data: `c2h_stream` (capture) and `iwfg_h2c` (transmit).
- **The workstation GUI** (`sdr_workstation`, Qt). It provides live and offline spectrum, waterfall, time-domain and constellation analysis, RF measurements, an IQ signal generator for transmit, RF data converter control and loopback verification. An optional RoCEv2/GPU data path is included. A command-line client, `rfdc_ctl`, offers the same control functions as the GUI.

Package layout

Code	Folder	Contents
DF	<code>iW-ASCEJ-01-DF-R1.0-REL1.0</code>	Package root, release notes, API workbook, manifest
HF	<code>iW-ASCEJ-01-HF-R1.0-REL1.0</code>	PS configuration, Vitis platform export, hardware interface summary
FF	<code>iW-ASCEJ-01-FF-R1.0-REL1.0</code>	Firmware sources, BOOT.BIN recipe, firmware documentation
BN	<code>.../iW-ASCEJ-01-BN-R1.0-REL1.0</code>	Prebuilt host binaries
SC	<code>.../iW-ASCEJ-01-SC-R1.0-REL1.0</code>	GUI, helpers, kernel driver, host-tool sources
UM	<code>.../iW-ASCEJ-01-UM-R1.0-REL1.0</code>	This manual, topic guides, developer notes
AT	<code>.../iW-ASCEJ-01-AT-R1.0-REL1.0</code>	Acceptance test procedure and report
TS	<code>.../iW-ASCEJ-01-TS-R1.0-REL1.0</code>	Automated test suite and scripts

In this manual, paths are written with the folder code first. For example, `SC/01_sdr_workstation_GUI` means `iW-ASCEJ-01-SF-R1.0-REL1.0/iW-ASCEJ-01-SC-R1.0-REL1.0/01_sdr_workstation_GUI`.

Conventions

Convention	Meaning
<code>monospace</code>	Commands, file names, register and API names
Bold	GUI controls and labels
MiB/s	2 ²⁰ bytes per second (all DMA rates)
MSPS	Million complex samples per second
Menu <i>n</i>	Option <i>n</i> of the firmware's serial-console menu

Related documents

Document	Location
Release notes	DF <code>iW-ASCEJ-01-ReleaseNotes-R1.0-REL1.0.pdf</code>
API and engineering-calculation reference	DF <code>iW-ASCEJ-01-API-Reference-R1.0-REL1.0.xlsx</code>
Hardware interface summary	HF <code>iW-ASCEJ-01-Hardware-Interface-Summary-R1.0-REL1.0.pdf</code>
Acceptance test procedure / report	AT <code>iW-ASCEJ-01-ATP-R1.0-REL1.0.pdf</code> , <code>iW-ASCEJ-01-ATR-R1.0-REL1.0.xlsx</code>
BOOT.BIN build (Vitis 2025.2)	FF <code>03_Documentation/BOOT_BIN_2025.2.md</code>
Driver overflow-recovery analysis	UM <code>02_Developer_Notes/DRIVER_OVERFLOW_RECOVERY.md</code>

System overview

Signal chains

```

TRANSMIT  GUI IQ generator --FIFO--> iwfg_h2c --/dev/iwfg1 (QDMA H2C)--> AXIS switch S00
          --> RF-DAC (NCO, x24 interpolation) --> RF out
          DDS compiler --> AXIS switch S01 (on-chip test tone, boot default)

RECEIVE   RF in --> RF-ADC (x24 decimation, NCO/DDC, DSA, QMC) --> AXIS routing (ch0..3)
          --> PL GPIO channel select (1/2/4 ch) --> QDMA C2H --/dev/iwfg0--> c2h_stream
          --FIFO--> GUI DmaSource --> DspEngine (FFT, metrics, EVM) --> plots

CONTROL   GUI RFDC tab / rfdc_ctl --BAR 2--> RFDC_configuration (0xAC) + ring_register (0xA8)
          --> firmware pcie_cfg.c --> XRFdc / GPIO / switch --> ACK (0x2A<<24 | err<<20 | event)

```

Default operating point (firmware boot values)

Parameter	Value	Source
ADC sampling rate	4800 MSPS	PCIE_DEF_ADC_FS_MHZ
ADC decimation	$\times 24 \rightarrow$ 200 MSPS per channel	PCIE_DEF_DECIMATION
DAC sampling rate	9600 MSPS	PCIE_DEF_DAC_FS_MHZ
DAC interpolation	$\times 24$	PCIE_DEF_INTERPOLATION
DAC stream clock	200 MHz; one 32-bit I/Q word per clock = 762.9 MiB/s	clk_wiz_dac
NCO (ADC and DAC)	3100 MHz	PCIE_DEF_NCO_MHZ
DDS tone	10 MHz	PCIE_DEF_DDS_MHZ
ADC DSA	0 dB	PCIE_DEF_ADC_DSA_DB10
DAC VOP	32 mA	PCIE_DEF_DAC_VOP_UA
DAC input source	DDS compiler	PCIE_DEF_DAC_SOURCE
PL GPIO mode	3 (four channels)	PCIE_DEF_PL_GPIO_MODE
ADC AXIS routing	0x1237 (ch0=7, ch1=3, ch2=2, ch3=1)	PCIE_DEF_AXIS_ROUTING

Bandwidth facts that shape operation

Quantity	Value	Consequence
Capture, 4 ch $\times$ 200 MSPS $\times$ 4 B	3052 MiB/s	More than the measured host drain ($\sim$ 2400 MiB/s). The C2H FIFO overflows.
Capture, 2 ch $\times$ 200 MSPS $\times$ 4 B	1526 MiB/s	Fits the 1600 MiB/s transmit budget
DAC feed	762.9 MiB/s, continuous	The DAC input has no flow control, so a shortfall plays as interference
H2C chunk 1 MiB	99.7 % transfer efficiency	The default. The DAC is kept fed.
H2C chunk 16 KiB	$\approx$ 640 MiB/s	Starves the DAC
Host tone resolution	$f_s / (\text{chunk} / 4) = 762.9 \text{ Hz at } 1 \text{ MiB}$	The generator snaps to a coherent frequency

On a C2H overflow, the driver resets the whole QDMA, and the H2C queue is reset with it. For this reason the GUI's **transmit guard** limits the capture to 2 channels while the generator runs. Chapter "Capture and transmit together" explains this. The workbook's *Calculations* sheet computes all of these values live for other configurations.

Requirements

Target hardware

- iWave ZU47DR RFSoc board (xczu47dr-ffvg1517-2-i), with the PL design that matches `H2C_pcie_control.xsa`. That design has the QDMA, the `DDS_H2C_switch` and the ADC channel switch.
- A PCIe connection to the host. The PS reference clock is 33.333 MHz, and the GT reference for PCIe is 100 MHz (see the Hardware Interface Summary).
- A UART console on PS UART0 (MIO 6/7), 115200 8N1, for firmware messages and the menu.
- RF cabling. For the loopback checks, connect a DAC output to an ADC input through suitable attenuation.

Host computer

Item	Requirement
OS	x86-64 Linux. Ubuntu 24.04 was used for the prebuilt binaries.
Kernel headers	For building <code>iwfg.ko</code> (<code>linux-headers-\$(uname -r)</code>)
Toolchain	gcc/g++, CMake $\geq$ 3.16, make
Qt	Qt 5.15 (Widgets) or Qt 6
glibc	$\geq$ 2.38 for the prebuilt binaries in BN. Otherwise build from SC.
Optional	NVIDIA GPU + CUDA (GPU FFT path); Mellanox NIC (RoCEv2 path)

Firmware build

AMD Vitis 2025.2 (SDT flow) or 2022.2 (classic flow), with the platform built from the PL hand-off (.xsa). FF/02_Boot holds the BOOT.BIN recipe.

Installation

Install in this order: firmware first, then the driver, then the GUI, then verify. Each step has a check. Do not go to the next step until the check passes.

Step 1: verify the package

```
cd iW-ASCEJ-01-DF-R1.0-REL1.0
iW-ASCEJ-01-SF-R1.0-REL1.0/iW-ASCEJ-01-TS-R1.0-REL1.0/02_Scripts/verify.sh
```

Check: RESULT: PASS. The script verifies MANIFEST.sha256, the firmware source set (37 files) and the firmware ↔ GUI ↔ CLI contract.

Step 2: firmware

1. Create, or open, the Vitis application on the platform built from the PL hand-off (H2C_pcie_control.xsa). HF 02_Vitis_Platform/export.json records the reference workspace (zu47dr_plat, standalone_domain).
2. Copy **all 37 files** from FF/01_Source into the application's src/. Never copy a subset.
3. Merge FF/02_Boot/lscript_coeff_fragment.ld into the linker script if your application does not have the calibration-coefficient section yet.
4. Build. Then create BOOT.BIN:

```
cd FF/02_Boot
./make_boot.sh <vitis-workspace>      # --list shows what it will pick
```

boot.bif documents the partition order: FSBL, PMUFW, **bitstream before the application**, then the application at EL3. FF/03_Documentation/BOOT_BIN_2025.2.md explains every option and failure.

5. Program QSPI or SD, or boot over JTAG (jtag_boot_test.tcl). Then open the UART console (PS UART0, 115200 8N1).

Check: the console prints, among the init messages:

```
[init]    DAC input = DDS compiler
[pcie] pcie_cfg ABI 4, built <date>, caps 0x7F:
        basic extended start/stop READBACK(0x0F1) DEFAULTS(0x023) DAC-SOURCE-MAPPED
```

ABI 4 or higher is required. If the [pcie] line is missing, pcie_cfg.c is stale. Reflash all files together.

Step 3: QDMA driver (iwfg)

```
cd SC/01_sdr_workstation_GUI/drivers
make                # produces iwfg.ko for the running kernel
sudo insmod iwfg.ko # or: sudo make install && sudo modprobe iwfg
ls -l /dev/iwfg*    # expect /dev/iwfg0 (C2H) and /dev/iwfg1 (H2C)
dmesg | grep -i iwfg
```

The GUI can also do this for you. On a PCIe launch, its pre-flight checks the module, and its **Fix driver** action builds and inserts it through pkexec. On exit it removes the module only if that session loaded it.

Check: both device nodes exist. If C2H works while H2C fails at once, /dev/iwfg1 is missing.

Step 4: GUI and helpers

From source (recommended, about a minute):

```
cd SC/01_sdr_workstation_GUI
cmake -B build -DCMAKE_BUILD_TYPE=Release      # add -DSDR_ENABLE_CUDA=ON for the GPU path
cmake --build build -j
```

The build also compiles the backend helpers (backend/bin/c2h_stream, backend/bin/iwfg_h2c) and the UDP receiver for **this** host.

Prebuilt alternative: BN/01_Host_x86-64/sdr_workstation needs glibc 2.38 or newer (ldd --version). On an older system it stops with GLIBC_2.38 not found. That is expected; build from source instead.

The CLI:

```
cd SC/02_host_tools && make      # builds rfdc_ctl
```

Access to the PCIe BAR without sudo

The RFDC tab maps a PCIe BAR, and that needs privilege. Running the GUI as root breaks the GPU path, because a CUDA IPC handle cannot cross users. Grant the capability instead:

```
sudo setcap cap_sys_rawio+ep SC/01_sdr_workstation_GUI/build/sdr_workstation
```

Step 5: verify host-side (optional, no board needed)

```
TS/01_Host_Test_Suite/run_all.sh # 10 tests, 244 checks, ≈ 4 minutes
```

Check: ALL TESTS: PASS.

Quick start

First run in simulation (no hardware)

```
./build/sdr_workstation --skip-config
```

The simulator produces a modulated signal with a burst envelope. Spectrum, waterfall, time-domain and constellation views update at up to 30 fps. Open **H2C CONTROL** and set **DAC input = Host**. Start the generator: the simulated loopback shows the generator's tone. Then press **Verify loopback**.



First live capture

1. Power the board. Check the UART banner (installation step 2).
2. Start the GUI. In the configuration dialog, select **Live — PCIe DMA**, with `/dev/iwfg0` and format `cs16`. Set the channel count to match the PL GPIO mode (4 at boot), and the rate to **200 MSPS**.
3. The pre-flight shows the link discovery: driver, nodes, PCIe link speed and width. It continues automatically.
4. Press **Start**. The status bar shows the capture rate in MB/s. The default DDS tone (10 MHz, through the 3100 MHz NCOs) is visible if a DAC output is looped back to an ADC input.

First transmit loopback

1. Open **H2C CONTROL** ▶ **RFDC** tab ▶ **Attach**, then press **Ping**.
2. Set **DAC input source = Host / GNU Radio stream**. The chain view must say *confirmed by the device*.
3. Route the looped-back ADC to **AXIS output 0 (RFDC ▶ ADC Channel Routing)**. Output 0 stays visible when the transmit guard reduces the capture to 2 channels.
4. **Transmit** tab: choose a waveform and frequency, keep **Loop-coherent** ticked, then press **Start generator**. The **H2C to card** row should read about 762.9 MiB/s.

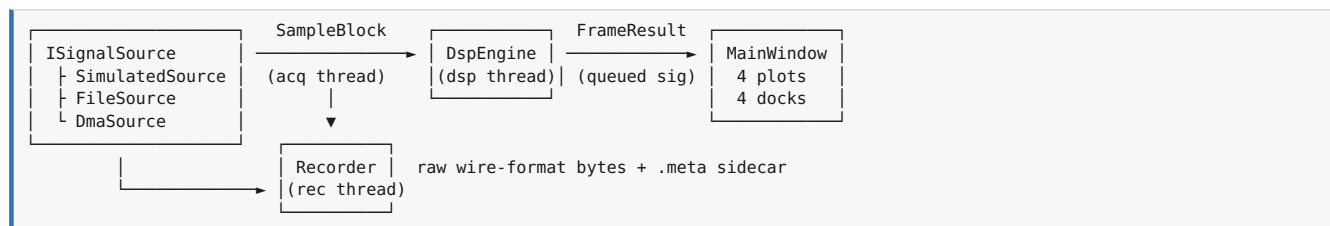
5. Press **Verify loopback**. The GUI moves the generator and checks that the received tone follows it.

The rows of the H2C TRANSMIT CHAIN view say what each stage is doing:

DAC input	Host / GNU Radio stream – confirmed by the device
DAC	9600.00 MSPS, interpolation $\times 24 \rightarrow$ stream 200.000 MSPS
NCO / RF	DAC NCO 3100.000 MHz $\rightarrow$ tone at RF ... · ADC NCO 3100.000 MHz
ADC (loopback)	4800.00 MSPS, decimation $\times 24 \rightarrow$ 200.000 MSPS per channel
H2C to card	iwfg_h2c is moving 762.9 MiB/s to the card (the DAC reads 762.9)
Received	... at ... dBFS – the generator's frequency

Workstation GUI operation

A Qt-based RF signal analysis workstation with two fully interchangeable acquisition paths — **live streaming from a PCIe DMA device node or FIFO**, and **offline replay of recorded captures** — feeding one shared DSP and visualisation pipeline.



Running

Launching with no arguments shows a configuration dialog. All of it can be bypassed from the command line:

```

./build/sdr_workstation --skip-config # simulator, defaults
./build/sdr_workstation --file cap.bin --format cs16 --channels 2
./build/sdr_workstation --device /dev/iwfg0 --format cs16 --channels 2 --rate 122.88
./build/sdr_workstation --skip-config --capture ui.png --capture-delay 5000
  
```

--capture renders the window to a PNG after settling and exits — combined with `QT_QPA_PLATFORM=offscreen` it gives reproducible headless screenshots for CI.

Live mode (PCIe DMA)

Select **Live — PCIe DMA / FIFO**, point *Device / FIFO* at your DMA character device or a named pipe, and press **Start**. The source:

- opens with `O_NONBLOCK` (a FIFO with no writer would otherwise block `open()` and freeze the UI),
- `poll()`s while idle and falls back to blocking reads on character devices that return `POLLNVAL`,
- accumulates partial reads into fixed-size blocks, so short DMA completions never corrupt sample framing,
- services Stop/Pause/config changes between read slices by re-posting its pump through the thread's event loop rather than spinning.

Try it without hardware:

```

mkfifo /tmp/iwfg0
python3 tools/make_test_capture.py /tmp/sig.bin --channels 2 --format cs16
cat /tmp/sig.bin > /tmp/iwfg0 & # stand-in for the DMA engine
./build/sdr_workstation --device /tmp/iwfg0 --format cs16 --channels 2
  
```

Adapting to real hardware

`DmaSource` in `src/core/Sources.cpp` reads a plain byte stream. If your fabric prepends per-transfer descriptors or uses ioctl-driven ring buffers, that file is the only place to change; everything downstream consumes `SampleBlocks` and does not care where the bytes came from.

Offline mode

Select **Offline — capture file**, browse to a recording, Start. Features: loop-at-end, paced replay at nominal rate (or any speed multiple, or free-run), sample-aligned seeking from the position slider, and a header byte skip for files with a fixed prefix.

Opening a file that has a `.meta` sidecar restores its sample rate, wire format, channel count, centre frequency, NCO, decimation and interpolation automatically. This applies to **every** route into offline mode — the browse button, the File menu, and the `--file` command line — so a capture always replays with the settings it was taken with. Without it, a file recorded at a decimated rate is replayed and down-converted against whatever happens to be on screen, which smears the constellation into concentric rings. Recordings made by the app always carry one; `tools/make_test_capture.py` writes one too.

Data types

Named after the GNU Radio stream types they correspond to. Complex formats interleave `ch0.I ch0.Q ch1.I ch1.Q ...` per sample-time, little-endian; real formats carry one scalar per sample and are lifted into the complex pipeline with `Q = 0` (their spectrum is conjugate-symmetric, which is a property of the data, not a display artefact).

name	tag	bytes / sample
Complex Float 32	fc32	8
Complex Float 64	fc64	16
Complex Int 32	sc32	8
Complex Int 16	sc16	4
Complex Int 12 (packed)	sc12	3
Complex Int 8	sc8	2
Complex Byte (uint8)	uc8	2
Float 32	f32	4
Float 64	f64	8
Int 32	s32	4
Short 16	s16	2
Byte (int8)	s8	1
Byte (uint8)	u8	1

Format combos bind the enum as item *data*, not row index, so the list can be reordered or extended without silently reinterpreting a saved configuration.

Captures hold **post-DDC samples at the decimated (display) rate**; the configured ADC rate and decimation exist so the axes and paced replay know what that rate is. The recorder writes the same layout it received, so a recording replays byte-identically.

Signal processing

- **DDC**: complex NCO mix, phase derived from the absolute sample index so display-throttled (dropped) blocks cannot rotate the constellation.
- **Spectrum**: radix-2 FFT, 7 windows (Rect/Hann/Hamming/Blackman/ Blackman-Harris/Flat-top/Kaiser) with per-window coherent gain correction so a full-scale tone reads 0 dBFS under any of them.
- **Averaging**: off, exponential, linear-N, max hold, min hold. Linear-N averages in *linear power* — averaging decibels biases the result low.
- **Measurements** per channel: peak level/frequency (parabolic sub-bin interpolation), median-based noise floor, SNR, 99 % occupied bandwidth, channel power, RMS, peak amplitude, PAPR, EVM, phase error, DC offset, I/Q imbalance, and an N-deep peak table with delta-to-P1.
- **Constellation**: envelope-gated symbol capture at the symbol instant, static 4th-power phase alignment, density-graded persistence display.

Plots

All four plots share one interaction model:

action	effect
wheel	zoom X (Ctrl = Y, Shift = both)
left-drag	pan
Shift-drag / middle-drag	box zoom
Ctrl+click	drop marker (up to 4, with Δ readout)
double-click	reset view
right-click	context menu (markers, crosshair, reset)

Views are held in data units, so a zoomed region stays put while frames stream underneath. Spectrum and waterfall X axes are linked. The waterfall scrolls newest-first with the history axis counting downward.

Responsiveness

The pipeline is designed so the window stays interactive when the machine is saturated:

- **Render gate**. The DSP stage checks a shared atomic before posting a frame and skips while the GUI still owes a paint. Without it the engine outruns the compositor, and the queued-connection backlog monopolises the GUI thread's event loop and starves its timers.
- **Frame coalescing**. Frame arrival only marks state dirty; painting runs on a GUI-owned timer at the configured refresh rate, so a burst of frames costs one repaint rather than one repaint each.
- **Bounded analysis window**. A block at 122.88 MSPS holds a quarter-million samples per channel, but the display consumes one FFT and a 2048-point trace. Only a window is down-converted and measured, as a real analyser does. Throughput accounting and recording still see the full stream.
- **Waterfall ring buffer**. Scrolling advances a cursor and writes one row; paint stitches the ring with two blits. Previously every frame memmoved the whole image.
- **Worker threads run below the GUI thread**, so a saturated pipeline loses frames rather than the UI.
- **Envelope rendering for dense traces**. When a time-domain trace carries more than ~ 2 samples per pixel column, it is drawn as a per-column min/max envelope — antialias-free vertical lines, exactly as an oscilloscope renders. The naive antialiased polyline is catastrophically slow for high-frequency content (AA cost scales with the area each segment covers; a fast waveform makes every segment span the full plot height — measured at 0.6–2.4 s *per paint* in the field). The envelope is $O(\text{plot width})$ regardless of density and is more faithful: a decimated polyline aliases, min/max preserves the true extremes. The spectrum trace similarly takes the loudest bin per pixel column, which is what a swept analyser shows.

Simulator and sample rate

Sample rate drives span, RBW, the time base, and file replay pacing. The built-in **simulator** tracks the configured rate only up to a 16 k-sample block ceiling — above roughly 1 MSPS a software generator cannot synthesise in real time, and generating more than the DSP analyses only starves the GUI. The status panel therefore reports **measured** rate next to **configured** rate rather than pretending they match. File and DMA sources carry the true rate and are unaffected.

Hardware pre-flight and driver management

Selecting **GPU** in the launcher probes NVIDIA availability on the spot (driver, CUDA, model, memory) and falls back to CPU with a clear notice when no usable GPU is present.

Selecting **PCIe** for a live session runs a pre-flight before the main window opens: iwfg module presence and version, device-node existence and readability, PCI enumeration (vendor/device ID, current and maximum link speed and width, BARs), presented as a brief link-discovery screen that auto-continues on success. On failure the launcher stays open with the exact reason, and a "Fix driver" action drives the integrated driver manager: dependency check → out-of-tree make in `drivers/` → `pkexec insmod` → node re-validation. The module is inserted by exec'ing `insmod` directly under polkit — never via a shell with an interpolated path, and never silently. Drop the `iwfg_SDR_Driver` sources into `drivers/` to enable the self-contained build (see `drivers/README.md`).

Streaming-rate architecture

Sources publish **undecoded wire bytes**; the DSP stage decodes only after its display throttle and render gate, so at 800 MB/s roughly 3 % of blocks ever pay for conversion — previously the decode of every block pinned a full core. The recorder writes those raw bytes straight to disk, making recordings byte-identical to the stream at memcpy cost. Dense time-domain traces render as a per-column min/max envelope (O(width), antialias-free, as an oscilloscope draws) instead of an antialiased path whose cost scales with the area every segment covers; the displayed window is aligned to a rising zero-crossing each frame, so the trace holds still like a triggered scope instead of re-phasing every update.

Driver lifecycle (PCIe)

The application manages the iwfg module automatically and leaves the system clean on exit. Drop the validated `iwfg.ko` into `drivers/`; on PCIe launch the pre-flight inserts it directly (`insmod iwfg.ko`, source build only as a fallback), verifies load, and confirms the `/dev/iwfg*` node before enabling streaming. On exit the pipeline is stopped and descriptors closed first, then the module is removed (`rmmmod iwfg`) — but **only if this session loaded it**. A module you inserted by hand is left untouched. No Xilinx memory-controller node is created or expected; it is absent by design in this architecture.

Channel selection (1-16)

The live panel exposes a channel matrix: set the wire channel count (1-16), then tick any subset to process and display — a single channel, a pair, a contiguous range (1-8), or a scattered set (CH1, CH4, CH7, CH10). Presets cover All / None / Even / Odd / 1-8. The selection is a bitmask carried in `SourceConfig::channelMask`; the DSP processes only the selected channels, so picking 1 of 16 costs a sixteenth of the work rather than all-then-discard. The mask is saved in the record sidecar and restored on replay.

Ethernet (Mellanox)

Selecting the Ethernet interface detects the first Mellanox adapter (PCI vendor 0x15b3) and applies the lab configuration — 192.168.1.1, MTU 9000, link up — via `ip` under polkit, then reports interface, MAC, link state, MTU and IPv4 in the launcher. A system without a Mellanox NIC gets a clear status line instead of an error.

Plot navigation

Every plot (time domain, spectrum, waterfall, constellation) supports:

action	effect
wheel	zoom X (Ctrl = Y, Shift = both) — unlimited in both directions
left-drag	pan
Shift-drag / middle-drag	box zoom
double-click	auto fit (resume following the data)
right-click	auto fit · zoom in · zoom out · set axis range... · markers · crosshair
Ctrl+click	drop marker (up to 4, with delta readout)

Zoom is bounded only by double precision (span 1e-9 ... 1e15), so an arbitrary span can be held indefinitely. Earlier builds snapped the view back to the data extent on zoom-out *and* re-armed auto-fit, which dragged the user back to fit-to-screen on the next frame; the override is now set explicitly by the act of zooming and is only cleared by an explicit auto-fit.

"Set axis range..." accepts an exact `min`, `max` pair for work that needs a specific window rather than whatever scrolling lands on.

Channel selection

The **Select** field in the source panel shows the current selection (1,2,5,8 or All (16)) and opens a modal chooser with one checkbox per channel, plus Select All / Clear All / OK / Cancel. Cancel preserves the previous selection; OK is disabled while nothing is ticked. The selection is a bitmask (`SourceConfig::channelMask`) applied in the DSP, so only the chosen channels are processed and plotted, and it is saved to the record sidecar and restored on replay.

Display scaling

High-DPI scaling is enabled before `QApplication` is constructed (set afterwards, the attributes are silently ignored) with `PassThrough`

fractional rounding. Row heights, stepper buttons and status widgets are sized from font metrics rather than literal pixel counts, so text does not clip at scaling factors above 1.0.

Backend integration (live acquisition)

Live mode runs the supplied driver applications **unmodified** and attaches the workstation's existing `DmaSource` to their FIFO. The workstation does not re-implement DMA, buffer registration or streaming synchronisation — the validated applications do that exactly as they do under GNU Radio.

Pressing **Start** in live mode shows a mode popup:

mode	processes launched	data path
C2H	<code>c2h_stream <dev> <fifo></code>	card → FIFO → workstation
H2C_C2H	<code>iwfg_h2c then c2h_stream</code>	FIFO → card → FIFO → workstation

Playback starts before capture in loopback mode, so the card is consuming before the capture side attaches.

Architecture

`src/core/BackendLauncher.{h,cpp}` owns the process lifecycle and contains **no UI code** — it reports everything through Qt signals (`statusMessage`, `logLine`, `becameReady`, `startFailed`, `processFailed`, `stopped`), so the UI layer renders diagnostics however it likes and the backend is testable headlessly. The only frontend change is the mode popup and the Start/Stop bridge; every existing screen, widget and interaction is untouched.

- **Non-blocking.** `start()` returns immediately; readiness arrives by signal.
- **Graceful stop.** SIGINT first — both applications install handlers that unwind the DMA path and close the device cleanly — escalating to SIGTERM then SIGKILL only on timeout.
- **No orphans.** The destructor stops everything; an orphaned capture app would keep `/dev/iwfg0` open and break the next run.
- **Failure recovery.** A stage dying unexpectedly is reported with its exit code and the app's own stderr text, and the rest of the pipeline is torn down rather than left half-running.
- **Auto-build.** Helpers are built on first use via `backend/Makefile`.

`backend/` holds the application sources (`C2H/fifo_pipe.c`, `H2C/iwfg_h2c.c`) and their headers; `make` there produces `bin/c2h_stream` and `bin/iwfg_h2c`. The GNU Radio reference flows are kept alongside for comparison. The RF-parameter module is untouched.

Back-pressure and integrity

Source and DSP share an atomic in-flight block counter. When it reaches capacity the source **drops the block and counts the dropped samples** — the same overrun semantics a real DMA ring gives you — and the status panel shows buffer depth, drops, and display frames throttled separately, so "the GUI is skipping frames" is never conflated with "samples were lost".

Layout

Four dockable panels (source & signal chain, display & analysis, acquisition status, measurements) around a 2×2 plot grid; every dock floats, moves, or closes, plot visibility is toggleable, four layout presets are built in, and geometry persists across runs via `QSettings`.

Hardware integration

GPU detection

Selecting **GPU offload** in the launcher probes the machine immediately: `nvidia-smi` (2.5 s hard timeout) for model, memory, driver version and compute capability, `/proc/driver/nvidia/version` for the driver, and the CUDA runtime library/toolkit on disk. Success shows the full summary; any failure states the reason and falls back to CPU automatically, so a mode that cannot work is never launched.

PCIe pre-flight and link discovery

Whenever a session starts in PCIe mode, a ~1 s initialization screen validates the stack before the workstation opens: PCIe link status, current vs max link speed and width, device enumeration (BDF), vendor/device IDs, BAR regions, `iwfg` kernel module + version, device node accessibility, and DMA engine readiness — all read from `sysfs/procfs` without privileges. On success it transitions automatically; on failure it stays up with a one-line reason instead of letting acquisition start against a broken configuration. A named FIFO passes with only the node check, so loopback testing needs no hardware. Headless capture of this screen: `set SDR_CAPTURE_DISCOVERY=1 with -capture`.

Integrated driver management (`drivers/`)

The package carries the `iwfg` driver in `drivers/` (see the README there for what to drop in). When PCIe pre-flight finds the module missing and sources are bundled, the discovery screen offers **Build & load driver**, which runs the chain: dependency check (`make`, compiler, kernel headers — each missing one reported with its install command) → out-of-tree `make` → `pkexec insmod` → node re-validation, with per-step diagnostics on failure. Without a polkit agent the exact `sudo insmod` command is shown instead — the application never escalates privileges silently.

Tools

`tools/make_test_capture.py` — synthesises multi-channel QPSK/16-QAM/64-QAM or noise captures in any supported wire format, with burst gating, carrier offset, AWGN and a matching `.meta` sidecar. Needs NumPy.

RF data converter control (RFDC tab)

iWave SDR Analysis Tool · **H2C CONTROL** window · **RFDC** tab

1. What this tab controls, and what it does not

The window carries **two independent control paths**. Knowing which is which saves a lot of confusion:

Tab	Talks to	Over	Controls
Transmit	rdma_tx on this PC	/dev/shm/iqctl	pacing, sample rate, centre frequency
Signal chain	this PC	in-process	display/session values
RFDC	bare-metal firmware on the ZU47DR	PCIe BAR registers	the converters themselves

The Transmit tab paces the host-side stream. **The RFDC tab configures the silicon that stream lands in.** Changing one does not change the other.

2. First-time setup

2.1 Prerequisites

1. **Firmware must be current.** The board needs the build containing events `0x023` (defaults) and `0x0F1` (readback). If it doesn't, the status line shows `ring = 0x2A4xxxxx` — error 4, *unknown event code* — and Reset and Read will not work.
2. **Run the GUI as root**, or grant `CAP_SYS_RAWIO`. Mapping a PCIe BAR through sysfs is privileged.
3. The bare-metal application must be at its **main menu**. If the console is sitting at a sub-prompt such as `Enter ADC Tile (0-3):`, the CPU is blocked in `scanf` and will not poll PCIe. Press Enter on the serial console.

2.2 Attach

Field	Value	Why
Endpoint (BDF)	e.g. 0000:3c:00.0	Auto-populated with Xilinx (10ee) devices
BAR	2	BAR2 is the 4 KB control window. BAR0 is QDMA control space — writing register offsets there can disturb the iwf driver
Register base	0x0	Only non-zero if the block sits inside a larger window

Press **Attach**. Green means mapped.

Message	Meaning
Cannot open ... (needs root or CAP_SYS_RAWIO)	Not privileged
a driver holds this BAR exclusively	Use the character-device backend
No such BAR	Wrong BDF — check lspci -D

2.3 Prove the path — press Ping

Ping changes **no hardware**, so success proves the whole chain: host write → PCIe → firmware poll → decode → acknowledge.

```
ring = 0x2A0000F0    cfg = 0x00000000    NEW CMD=0    ACK ok
```

If Ping fails, nothing else on this tab will work. Fix it first.

3. Reading the status line

The line under the buttons is the live register state and is worth learning:

```
ring = 0x2A0000F0    cfg = 0x00989680    NEW_CMD=0    ACK ok
```

↑ ↑ ↑ ↑
└─ [23:20] error code payload of the last command 0 = idle, 1 = command pending
└─ 0x2A = this is a firmware acknowledgement

You see	Meaning
NEW_CMD=1 persisting	Firmware is not polling — check it is at the main menu
ACK ok	Last command applied
ACK err=0x4	Unknown event — firmware is out of date
ACK err=0x5	Payload out of range

ACK err=0x6	Event not valid for the selected converter
ACK err=0x7	That tile/block is not built in this design

4. Target selection — read this before changing anything

Converter / Tile / Channel apply to most commands below them. Setting NCO with *Tile 2, Channel 0* affects only that block.

Control	Meaning
Converter	ADC, DAC, or both
Tile	All tiles, or 0-3
Channel	Both, 0, or 1. ADC → blocks 0/1; DAC → blocks 0/2

Some commands are **global** and ignore tile/channel: sampling rate, Nyquist zone, AXIS routing, PL GPIO routing, DAC source, MTS, Start, Stop, Reset.

5. The controls

5.1 Immediate controls — apply on edit

Leaving the field (or picking from the combo) sends the command at once.

Control	Range	Notes
Decimation	1-40 (listed)	ADC only. Also triggers MTS re-align
Interpolation	1-40 (listed)	DAC only. Also triggers MTS re-align
NCO frequency	±10 GHz	Sent in kHz. Freezes/unfreezes TSCB around the change
NCO phase	−180...180°	Integer degrees
ADC DSA	0-27 dB, 0.5 steps	0 dB = maximum gain. Watch for clipping
QMC gain	0-1.9999	
DAC VOP	2.25-40.5 mA	Boot default 32 mA
DAC input source	DDS / host stream	Which feeds the DAC
DDS frequency	0-75 MHz	PL DDS compiler

5.2 Deliberate controls — need their own Apply

These are heavy or structural, so they are not sent on every keystroke.

Sampling rate — reprograms the PLL, re-aligns MTS, restores NCOs. Takes several seconds. See §6, this one has a trap.

Nyquist zone — forces zone 1 or 2, overriding automatic selection.

AXIS routing — four values, each 0-7, mapping an input stream to each output. Boot default is **7, 3, 2, 1**.

PL GPIO routing — modes 1/2/3 = 1/2/4 channels in PL logic. Boot default **Mode 3**.

5.3 Actions

Button	Effect
Ping	No-op handshake. Always safe
Read registers	Refresh the status line only
Read from hardware	Populate every field from the device
Start / Stop	Enable / disable all converter FIFOs. Stop means no data at all until Start
Re-align MTS	Re-run multi-tile sync and restore NCOs
Soft reset	MTS + NCO + routing restore. Not a converter power cycle
Reset all to defaults	Restore the firmware's boot configuration, then read it back
Abort current command	Appears while busy; stops waiting for the ACK

6. Sampling rate — the one real trap

The PLL cannot produce arbitrary frequencies. It synthesises $F_s = 300 \text{ MHz} \times \text{FBDIV} / \text{OutDiv}$, so most requests are quantised:

You ask for	You get	Error
2000 MHz	2006.25	+6.25
2400 MHz	2400.00	exact
3000 MHz	3000.00	exact
4000 MHz	4012.50	+12.5

4800 MHz	4800.00	exact
5000 MHz	4987.50	−12.5
6000 MHz	6000.00	exact
9600 MHz	9600.00	exact

Prefer 2400 / 3000 / 3600 / 4800 / 6000 / 9600 MHz and the error vanishes.

After a successful Apply rate the panel **re-reads the device automatically**, so the field settles on the *achieved* value. Ask for 2000 and you will see it change to 2006 — that is correct behaviour, not a glitch. The serial console prints the same thing:

NOTE: PLL quantised the request. asked 2000.000 MHz, achieved 2006.250 MHz.

Older firmware stored the *requested* rate and derived the stream clock, the Clocking Wizard divider, the DDS phase increment and the Nyquist zone from it — so everything downstream was quietly off. Current firmware uses the achieved rate throughout.

7. Recommended order of operations

Commands are not commutative. This order leaves the datapath consistent:

1. Sampling rate ← changes everything downstream
2. Decimation / Interpolation
3. Re-align MTS ← automatic after 1 and 2
4. NCO frequency / phase
5. DSA, QMC, VOP
6. AXIS / GPIO routing, DAC source
7. DDS frequency ← LAST: derived from the DAC stream clock

DDS goes last because its phase increment is computed from the DAC stream clock, which any interpolation or sampling-rate change moves.

8. Getting back to a known state

Press **Reset all to defaults**. One firmware command restores the boot configuration and the panel then reads it back:

Parameter	Default
ADC / DAC sampling rate	4800 / 9600 MHz
Decimation / Interpolation	×24 / ×24
ADC & DAC NCO	3100 MHz
DDS	10 MHz
ADC DSA	0 dB
DAC VOP	32 mA
DAC source	DDS compiler
AXIS routing	7, 3, 2, 1
PL GPIO	Mode 3
Nyquist	automatic

These match a power cycle exactly — the firmware is the authority, the GUI just displays it.

If this returns `ACK err=0x4`, the board's firmware predates the defaults command. Flash the current build.

9. Responsiveness

Every device transaction runs on a **worker thread**; the GUI never blocks — plots keep updating and the window stays usable throughout.

- A thin progress bar appears while a command is in flight.
- Device controls disable so a second command cannot queue behind a 15 s PLL reprogram.
- **Abort** stops waiting. The device may still finish the command; the log says so rather than pretending otherwise.
- **Read from hardware** is a single batched pass, not 13 round trips.

If the whole application freezes for seconds at a time, you are running an older build — that was a real defect and it is fixed.

10. Troubleshooting

Symptom	Cause	Fix
Everything times out	Firmware not polling	Press Enter on the serial console to leave any sub-menu
ACK err=0x4	Firmware out of date	Flash the current build
Fields don't update on Read	Readback unsupported by that PL build	Log says so; values shown are the last sent

NEW_CMD=1 stuck	Previous command never consumed	Check the board is running; reattach
Rate reads back different	Normal — PLL quantisation	Use an exact rate (§6)
No data after Stop	Working as intended	Press Start
EBUSY on attach	A driver owns the BAR	Use the character-device backend
Attach fails, not root	sysfs BAR needs privilege	Run with <code>sudo</code>

Cross-checking against the CLI

`rfdc_ctl` speaks the same protocol and is useful for isolating GUI from firmware:

```
sudo ./rfdc_ctl -b 0000:3c:00.0 -r 2 -v ping
sudo ./rfdc_ctl -b 0000:3c:00.0 -r 2 dump
```

If the CLI works and the GUI does not, the problem is the GUI. If both fail, it is the link or the firmware.

11. Safety notes

- **ADC DSA 0 dB is maximum gain**, applied to every block at boot. Combined with DAC VOP at 32 mA this maximises ENOB — and clipping risk on an unknown input.
- **Soft reset is not a power cycle**. It deliberately does not re-run the LMK/LMX clock-tree sequence, because dropping the clock tree from a remote host is not something a button should do.
- **Stop really stops**. No samples at all until Start.
- Sampling-rate changes tear down and rebuild the datapath. Do not change them while recording.

12. Device-state synchronisation (how the panel now behaves)

The panel follows one rule:

Read the device first, then update the software and the UI to match it.

12.1 On connect — automatic, and the UI waits

Pressing **Attach** runs a single sequence on the worker thread:

```
Attach (open + mmap)
  ↓
Ping                    ← prove the link before trusting anything read over it
  ↓
Read ALL registers
  ↓
Validate + update internal state
  ↓
Populate the panel
  ↓
Enable user interaction
```

Every control stays disabled until this finishes. That is deliberate: it is the only way a widget's default value cannot be written over a live device setting before you have seen what the device actually holds.

While it runs you see "*Synchronising with device...*" and a progress bar.

Outcome	What you see
Success	Green " <i>Synchronised with device.</i> ", controls enabled
Ping failed	Amber, controls stay disabled — the link is not trustworthy
Readback unsupported	Amber warning: values shown are not confirmed against hardware

Reconnecting re-runs the whole sequence. So does Complete Reset.

12.2 On change — write, read back, verify

Sampling rate uses the full loop:

Write → Read back → Compare → Update UI (with the DEVICE's value)

- **Match** → green confirmation, the field shows the confirmed value.
- **Quantised within 25 MHz** → still success, and it says so: "*Confirmed 2006 MHz (PLL quantised the 2000 MHz request by 6 MHz).*"
- **Outside tolerance, or read-back failed** → a warning dialog listing requested vs. actual. **The UI does not claim success.**

The 25 MHz tolerance exists because the PLL synthesises $\text{RefClk} \times \text{FBDIV} / \text{OutDiv}$ and legitimately cannot hit every request (§6). Demanding an exact match would report a failure where the hardware behaved correctly.

If readback is unsupported by the firmware, the write is reported as **not verified** rather than as success.

12.3 COMPLETE RESET

The red **COMPLETE RESET** button is deliberately separated from the ordinary controls, because it changes many settings at once.

It asks first:

Restore every RF Data Converter parameter to its default state? This will stop the datapath, restore the firmware's boot configuration (4800/9600 MHz, ×24, NCO 3100 MHz, DDS 10 MHz, 32 mA VOP, routing 7/3/2/1), restart the datapath and re-read every register.

Then it runs:

```
Stop datapath
↓
Restore firmware defaults
↓
Wait for the device to answer again (up to 2 s, polled by ping)
↓
Restart datapath
↓
Re-read ALL registers
↓
Report
```

Success is reported only if every stage succeeded. If defaults fail, the datapath is restarted anyway — leaving the converters stopped after a failed reset would be worse than the reset failing — and the dialog says the device may be in a partially reset state.

12.4 What is locked, and when

The panel cannot issue conflicting operations:

Phase	Controls
Before attach	disabled
Syncing	disabled, "Synchronising..." shown
Command in flight	disabled, progress bar, Abort available
Idle and attached	enabled

12.5 Error handling

Nothing is silently ignored:

Condition	Behaviour
Device disconnected	Attach fails with the reason; controls stay disabled
Register read failure	Sync reports failure; values are not shown as confirmed
Register write failure	Verification dialog; UI not updated as successful
Read-back mismatch	Warning dialog with requested vs. actual
Reset timeout	"Device did not become ready after reset."
Readback unsupported	Amber warning; values marked unconfirmed
Communication timeout	Logged, plus Abort to stop waiting

13. Acceptance checklist

Register synchronisation

- ☒ All registers read during initialisation
- ☒ Panel displays actual register values
- ☒ Default widget values cannot overwrite device values — UI is gated
- ☒ Sync runs again after reconnect
- ☒ Sync runs again after reset

Sampling rate

- ☒ Correct register only (ring EVENT 0x010, payload MHz)
- ☒ No unrelated register touched — one write per command
- ☒ Unit conversion documented and tested (MHz → payload, kHz for NCO)
- ☒ Written value read back and verified
- ☒ Panel displays the confirmed hardware value
- ☒ Quantisation reported rather than hidden

Complete reset

- ☒ Dedicated, visually separated control
- ☒ Confirmation dialog before executing
- ☒ All settings restored to firmware defaults
- ☒ Registers re-read afterwards
- ☒ UI synchronised with the result
- ☒ Success **and** failure clearly reported

Reliability

- ☒ Communication failures handled
- ☒ Never reports success when an operation failed
- ☒ Conflicting operations impossible during sync/reset
- ☒ Verified mechanically: no device call outside a queued worker lambda

H2C CONTROL window

A step-by-step guide to the **H2C CONTROL** window: opening it, what each control does, and how to drive the ZU47DR RF Data Converter (RFDC) entirely from the GUI. No terminal commands are needed at any point.

If you ever get lost, click **Reset all to defaults** on the RFDC tab — it puts every field back to a known baseline in one click.

1. Start the application

Launch the app. You do **not** pass anything on the command line.

1. The launcher dialog appears. Pick your source (Simulated / File / PCIe / Ethernet), the data-flow mode (ADC Only / DAC Only / ADC + DAC) and press the launch button.
2. The main workstation window opens.

That is all the setup there is. Everything below happens inside the GUI.

2. Open the H2C CONTROL window

On the main window toolbar, click **H2C CONTROL**.

- The window opens as a **separate, floating panel** — the main window stays fully usable behind it.
- It is available in **every** data-flow mode. In *ADC Only* mode the Transmit tab is inactive (there is no transmit path), but the **RFDC** tab still works, because converter configuration is valid regardless of data flow.
- Closing the window only hides it; your settings are kept and reappear when you open it again.

The window can be **resized** by dragging any edge or the corner grip, and each tab **scrolls**, so the lower controls are always reachable even on a small screen.

The window has three tabs: **Transmit**, **Signal chain**, and **RFDC**.

3. The RFDC tab (main converter control)

This tab talks to the bare-metal application on the board over the PCIe control registers. It is organised top-to-bottom in the order you normally use it.

3.1 Connect — "PCIe control interface"

Field	What it is
Endpoint (BDF)	The PCIe device address, e.g. 0000:3c:00.0. Xilinx cards (vendor 10ee) are listed automatically — pick one, or type your own.
BAR	The memory window that holds the control block. Default 2.
Register base	Offset of the control block inside the BAR. Leave at 0x0 unless your design differs.
Attach	Maps the registers and turns on all the controls below.

Press **Attach**.

- On success the status line turns green (Attached: ...) and every control on the tab becomes usable.
- On failure the status line turns red with the reason. Common cases are in the Troubleshooting table (§6).

Until you attach successfully, all the parameter and action controls stay disabled — so you can never send a command into thin air.

3.2 Choose the target — "Target"

These three selectors decide **what** the next parameter you edit applies to:

- **Converter** — ADC, DAC, or ADC + DAC.
- **Tile** — All tiles, or a specific Tile 0-3.

- **Channel** — Both channels, or Channel 0 / Channel 1.

Set these **first**, then edit a parameter.

3.3 Set parameters — "Parameters — applied on edit"

Important: each value is sent to the board **the moment you finish editing it** — press *Enter* or click away from a number box, or pick an item in a drop-down. There is no "apply everything" button, so **changing one field changes only that one thing** on the hardware.

Control	Meaning	Applies to
Decimation	ADC decimation factor (only the accepted factors are listed).	ADC
Interpolation	DAC interpolation factor.	DAC
NCO frequency	Digital mixer frequency, in MHz (may be negative).	Target
NCO phase	Mixer phase offset, $-180...180^\circ$.	Target
ADC DSA	ADC step attenuator, 0...27 dB. 0 = leave unchanged.	ADC
QMC gain	Gain correction. 0 = leave unchanged.	Target
DAC VOP	DAC output current, in mA. 0 = leave unchanged.	DAC
DAC input source	DDS compiler, or the host / GNU Radio stream.	DAC
DDS frequency	On-chip DDS tone frequency, in MHz.	DDS

A change to Decimation or Interpolation automatically re-aligns MTS afterwards, because it moves the stream clock.

3.4 Heavy / structural operations — "PLL, Nyquist & routing"

These are **not** applied on edit. You set the value, then press that row's own **Apply** button, so they are only issued deliberately:

- **Sampling rate** → *Apply rate*. Reprograms the PLL and re-aligns MTS — this one takes a few seconds, that is normal.
- **Nyquist zone** → *Apply zone*. Zone 1 or Zone 2 for the target converter.
- **AXIS routing** (four boxes) → *Apply*. Source stream for each AXIS output.
- **PL GPIO routing** → *Apply*. Routing mode 1–3.

3.5 Actions

Button	What it does
Ping	Handshake test. Changes nothing on the hardware, so a success proves the whole path end-to-end. Use it first after Attach.
Read registers	Refreshes the <code>ring / cfg</code> readout at the bottom.
Start	Enables the converter FIFOs (data flows).
Stop	Disables the FIFOs (no data until Start).
Re-align MTS	Re-establishes multi-tile sync.
Soft reset	<i>Firmware</i> reset: MTS re-align + NCO/routing restore. Does not power-cycle the converters.
Reset all to defaults	<i>Panel</i> reset — see §4.

3.6 The register readout

The bottom line shows the two live control registers:

```
ring = 0x.....   cfg = 0x.....   NEW_CMD=0
```

`NEW_CMD=0` means the board has consumed the last command and is ready for the next. If it sticks at `1`, see Troubleshooting.

4. "Reset all to defaults" — recover a known baseline

This button is the quick way out of a messy configuration, so you never have to re-enter every field by hand.

When you click it:

1. **Every control on the RFDC tab** is set back to its default value (Target = ADC, Tile = All, Channel = Both, Decimation/Interpolation = x1, NCO = 3100 MHz, phase = 0, DSA/QMC/VOP = 0, DAC source = DDS, DDS = 10 MHz, Sampling rate = 2000 MHz, Nyquist = Zone 1, AXIS = 0/1/2/3, PL GPIO = Mode 1).
2. **If a device is attached**, it also applies the default signal chain in one go — decimation, interpolation, MTS re-align, NCO frequency, NCO phase, DAC source and DDS — so the board itself returns to that baseline.
 - The 0-valued gain/attenuator fields (DSA, QMC, VOP) mean "leave unchanged", so they are not pushed.
 - Sampling rate, Nyquist and routing are heavy/structural, so those are reset in the panel but you apply them yourself with their own buttons if you want.
3. **If no device is attached**, it just resets the on-screen values, and the log says so.

Soft reset vs. Reset all to defaults:

- *Soft reset* asks the **firmware** to restore its own converter defaults.
- *Reset all to defaults* restores the **panel's** default values (and pushes the default signal chain). Use this one when you want the GUI and the board back to the baseline you started from.

5. The other two tabs

Transmit

Live control of the host transmitter. Three fields — **Sample rate**, **Centre frequency**, **Pacing** — plus:

- **Apply** — writes the values and bumps the sequence; the transmitter picks them up on its next poll.
- **Revert** — reloads whatever is currently in the control block.

Tick **Unlimited** to send unpaced. This tab is inactive in ADC-Only mode.

Signal chain

Describes the intended H2C transmit feed (NCO/DUC, interpolation, decimation, channels, wire format, chunk size). In the current build these are informational — the values the transmitter actually honours live are the three on the **Transmit** tab.

6. Troubleshooting

Symptom	Cause / fix
"No Xilinx PCIe endpoint found"	No card enumerated. Check the board is up and the link is trained.
"needs root or CAP_SYS_RAWIO" on Attach	The BAR needs privilege to map. Launch the app with the rights your setup uses for hardware access.
"a driver holds this BAR exclusively"	Another driver owns BAR 2. Detach it, or use the character-device backend for that stack.
Controls are greyed out	You are not attached. Fill in the endpoint and press Attach .
NEW_CMD stays 1	The previous command was not consumed — the firmware may be sitting in a UART sub-menu, or not polling. Press Read registers to re-check.
A value "did not take"	Make sure you committed it: press <i>Enter</i> or click away from the box, or re-pick the drop-down item. Watch the log line for the confirmation.
Sampling-rate change seems to hang	It reprograms the PLL and can take several seconds. That is expected.
Everything looks wrong	Click Reset all to defaults .

7. A first-time walkthrough

1. Open **H2C CONTROL** → **RFDC** tab.
2. Pick the **Endpoint (BDF)**, leave BAR = 2, press **Attach** → wait for green.
3. Press **Ping** → the log should say it applied. That proves the path works.
4. Set **Converter** = **ADC**, **Tile** = **All tiles**, **Channel** = **Both channels**.
5. Change **NCO frequency**, press *Enter* → the log confirms it was applied, and the `ring/cfg` line updates.
6. Press **Start** to enable the FIFOs.
7. When you want to begin again from scratch, press **Reset all to defaults**.

DAC input source and host transmit

1. What feeds the DAC

A 2-to-1 AXI4-Stream switch (`DDS_H2C_switch`) chooses what the DAC plays:

Value	Name	Source	Switch port
0	DDS compiler	internal tone, <code>dds_compiler_2</code>	S01_AXIS
1	Host / GNU Radio stream	samples sent over PCIe (H2C)	S00_AXIS

The port wiring comes from the bitstream itself: `design_1.hwh` inside `H2C_pcie_control.xsa`, instance `DDS_H2C_switch`.

Firmware **before 1.2.1** passed the value straight through as a port number, so 0 and 1 were swapped everywhere. The board also booted on the idle host stream, which meant no tone at power-on. 1.2.1 fixes this and boots on the DDS.

Selecting it

From	How
GUI	H2C CONTROL → RFDC → <i>DAC input source</i>
Serial	menu 16 → 0 DDS, 1 host (shows the current routing first)
CLI	<code>sudo ./rfdc_ctl -b <BDF> dac-source dds or ... dac-source host</code>

Every change is verified at the switch register. The GUI logs *"DAC input confirmed by the device: ..."*, and the serial menu prints *"(verified at the switch)"*.

How to tell which firmware you have

Serial console at boot:

```
[init] 8/9 DDS/H2C sw (PL 0x...)...
[init] DAC input = DDS compiler <- 1.2.1
[pcie] pcie_cfg ABI 4, ... caps 0x7F: basic ... DAC-SOURCE-MAPPED
```

Older firmware has neither line. The 1.2.1 GUI detects it and corrects for it, logging *"this firmware predates 1.2.1 ... The GUI is compensating"*. On that firmware, serial menu 16 and `rfdc_ctl dac-source` stay swapped until you reflash.

2. Transmitting from the host (IQ generator)

H2C CONTROL → **Transmit** → *H2C Signal Generator*.

1. Start the session with **C2H + H2C** (or *H2C DAC Only*). The GUI launches `iwfg_h2c`, which reads `/tmp/iwfg_h2c.fifo`. In an *ADC Only* session, it launches `iwfg_h2c` when you start the generator (1.2.4).
2. Attach the **RFDC** tab. The generator's *Sample rate* is then filled in from the DAC stream clock (200 MSPS at boot).
3. Pick the waveform, tone, IQ phase and amplitude, then press **Start generator**. If the DAC is on the DDS, the GUI switches it to the host stream and the device confirms.
4. The state goes from *Waiting for iwfg_h2c...* to *Transmitting* If it stays on *Waiting*, look at the **H2C playback** row of the chain, which says what the helper is doing and why.

You can change parameters while it runs; they take effect within about 20 ms.

Proving the samples reach the DAC (1.2.3)

The Transmit tab opens with the **H2C TRANSMIT CHAIN**. Each row is read from the device where possible:

```
DAC input      Host / GNU Radio stream – confirmed by the device
DAC            9600.00 MSPS, interpolation x24 → stream 200.000 MSPS
NCO / RF       DAC NCO 3100.000 MHz → tone at RF 3114.990 MHz · ADC NCO 3100.000 MHz
ADC (loopback) 4800.00 MSPS, decimation x24 → 200.000 MSPS per channel
H2C to card    iwfg_h2c is moving 762.9 MiB/s to the card (the DAC reads 762.9)
Received       -14.990 MHz at -40.2 dBFS – the generator's frequency (spectrum inverted)
```

Neither a busy DMA nor a tone on screen proves that the generator is what the DAC plays. The card accepts H2C data even when the switch drops it, and the tone could be the DDS. **Verify loopback** settles it. It moves the tone by 7.3 MHz and back, and checks that the received tone moves with it. On failure it names the cause:

Result	Meaning
✓ verified	the samples reach the DAC and come back through the ADC
✗ did not move — DAC input is the DDS	select Host / GNU Radio stream
✗ did not move — RFDC not attached	attach it (needs <code>cap_sys_rawio</code>), or serial menu 16 → 1; 1.2.1+ firmware boots on the DDS
✗ did not move — iwfg_h2c reports no transfers	the H2C backend is not running
✗ did not move, data moving, switch on host	the card-side path QDMA H2C → <code>H2C_data_in</code> → switch S00 is not delivering: probe with the ILA
moved by the wrong amount	the display's sample rate does not match the stream

The main window's live display rate is set from the device's ADC stream clock automatically, so received frequencies read in true Hz.

The helpers behind the chain, and how they recover (1.2.4)

Two rows show the helper applications themselves.

H2C playback is `iwfg_h2c`'s state, with the reason:

State	Meaning	What happens / what to do
waiting for the generator	running, nothing has connected to its FIFO	press Start generator
starting	the first buffer is being transferred	a working transfer reports a rate within about a second
streaming	the card is consuming	—
restarting	being relaunched with a smaller buffer, or by Retry transmit	automatic
stalled	the card is not consuming	see below
failed	<code>iwfg_h2c</code> exited	the reason is shown; press Retry transmit

Buffer size is automatic. The default is 1 MiB. If no transfer completes within 4 s of the first buffer, or `iwfg_h2c` reports stall kicks,

driver timeouts or device cycling before it has streamed, the GUI first **reopens it at the same size** (1.2.5). A reopen rebuilds the H2C queue and re-enables the PL H2C datapath, which the driver's C2H-overflow recovery can leave disabled. Only if the same size stalls again does it step down: 1 MiB → 256 KiB → 64 KiB → 16 KiB. The generator follows the new loop length by itself. A helper that has streamed and then stalls for about 4 s is also reopened at the same size, at most 3 times a minute.

If the card takes nothing even at 16 KiB, the problem is not the buffer. The row then reads *stalled — ... not a buffer-size problem*. Check:

- DAC input = Host (RFDC tab);
- the RF DAC is running (RFDC ▶ Start);
- the iwfg driver enables the PL H2C datapath (`dmesg | grep iwfg`).

`iwfg_h2c` stays running and resumes the moment the card consumes. When it does, the GUI relaunches it once at 1 MiB, so you are not left on 16 KiB.

C2H capture shows what the capture delivers against what the configured stream needs:

```
C2H capture 2417 MiB/s delivered for 3052 MiB/s needed (4 ch × 200.00 MSPS × 4 B)
- OVERLOADED (79%): the C2H FIFO overflows, and each overflow
  recovery resets the QDMA, which interrupts H2C transmit too.
```

An overloaded capture disturbs transmit. The driver recovers from a C2H FIFO overflow by soft-resetting the QDMA, and that rebuilds the H2C queue too, even mid-transfer. The details for the driver team are in `UM/02_Developer_Notes/DRIVER_OVERFLOW_RECOVERY.md`.

Transmit guard (1.2.5). When you start the generator in a live session, the GUI checks the capture. If it needs more than 1600 MiB/s (4 channels at 200 MSPS need 3052), the GUI:

- reduces it to 2 channels, or 1, via the PL GPIO mode (the RFDC tab must be attached);
- checks the measured rate afterwards, and follows the channel count the stream really carries;
- restores the original setting when the generator stops.

The channels kept are the first stream channels (AXIS outputs 0, 1). Route your looped-back ADC to AXIS output 0 on the RFDC tab (ADC Channel Routing) so it stays visible. Without RFDC access the GUI cannot change the PL mode, and says so in a notice.

To reduce the capture by hand for a loopback test, **Capture 1 channel (PL GPIO mode 1)** does it in one click (the RFDC tab must be attached): it sets PL GPIO mode 1 and the display to one channel. The channel kept is AXIS output 0; the ADC Channel Routing on the RFDC tab decides which ADC input that is. Three seconds later the measured rate is checked. If the stream did not shrink, both changes are undone and you are told.

Stop order and device reset (1.2.5). Stop closes transmit first and capture last. The driver soft-resets the QDMA only when the last C2H stream closes with no other stream open, so every Stop/Start now begins on a clean device.

If the capture delivers nothing for about 5 s while running (the card has stopped delivering C2H data), the GUI resets the device the same way: close transmit, close capture, start both again. It does this at most twice in 3 minutes. If that does not bring the data back, a notice tells you to reload the driver or power-cycle.

The capture FIFO also gets a larger pipe buffer (1 MiB instead of 64 KiB), so the host drains faster.

The capture helper restarts itself. `c2h_stream` exits with code 0 for every reason it stops, so the GUI reads what it printed instead. If it ends on its own after streaming, the GUI:

- names the reason (DMA driver error, reader closed, stray signal);
- restarts it, up to 3 times a minute;
- re-attaches the display to the new FIFO.

There is no dialog, only a status-bar line and a log entry. If the transmit helper was stalled at that moment, it is stopped too, because it is the likely culprit; Retry transmit brings it back. A capture helper that cannot start, or keeps dying, ends the session with the reason.

Everything the helpers print is written to `$TMPDIR/sdr_backend.log` (normally `/tmp/sdr_backend.log`), with timestamps. The previous session is kept as `.1`. Include this file when reporting a problem.

You see	Likely cause	Do
H2C playback <i>waiting for the generator</i> while the generator runs	the generator writes a different FIFO	it follows the backend's since 1.2.4; check the FIFO field
<i>restarting</i> repeatedly, then <i>stalled</i> at 16 KiB	the card consumes nothing	DAC input = Host, RFDC ▶ Start, driver
<i>streaming</i> , but Verify says the tone did not move	data reaches the card, not the DAC	FPGA path QDMA H2C → H2C_data_in → switch S00 (ILA)
C2H capture <i>OVERLOADED</i>	more channels than PCIe carries	Capture 1 channel
status bar <i>Capture helper ended — ... restarted</i>	a C2H DMA error (see the log)	nothing, if it recovers; otherwise send <code>sdr_backend.log</code>
status bar <i>Resetting the DMA device</i>	the capture delivered nothing for 5 s	nothing, if it recovers; after two tries the GUI asks you to reload the driver
<i>Transmit guard: capture reduced from 4 to 2 channel(s)</i>	the capture would overflow and disturb transmit	route the looped ADC to AXIS output 0 or 1
ADC row <i>QMC gain reads 0.0000</i>	if QMC gain is enabled, the capture is muted	RFDC tab: QMC gain 1.0 for the ADC

Keeping the DAC fed (1.2.2)

The DAC reads one 32-bit I/Q word per stream clock, which is 762.9 MiB/s at 200 MSPS. Its input has **no flow control**: if the host delivers less, the DAC plays stale data. On the spectrum that shows as a raised skirt around the tone, bursts across the whole band, and a dotted line at 0 Hz.

The generator panel shows the live figure under the state line:

```
H2C link 758.0 MiB/s for a DAC that reads 762.9 MiB/s – kept full (buffer 1 MiB).
△ H2C link 640.6 MiB/s, but the DAC reads 762.9 MiB/s: it runs dry about 16% ...
```

What makes the difference is the H2C buffer, `iwfg_h2c`'s chunk size. The helper sends one blocking DMA call per chunk. At 16 KiB, the fixed cost of each call held delivery at about 640 MiB/s (the red line above).

1.2.2 went to 16 MiB, and on the bench nothing then reached the DAC. A 16 MiB user buffer is thousands of H2C descriptors per transfer, and the card's H2C path did not complete it. From 1.2.4 the default is **1 MiB**, which already makes the per-call cost negligible (0.3 %). The GUI steps down by itself if even that does not work (see above). To force a size yourself:

```
IWFG_H2C_CHUNK_BYTES=4194304 ./SC/01_sdr_workstation_GUI/build/sdr_workstation
```

A steady, faint line at 0 Hz that does not flicker is the ADC's own DC offset, not the transmit path.

Why the tone can be a few Hz off what you typed

`iwfg_h2c` replays the chunk it last received, over and over. So the generator builds **one seamless loop exactly one buffer long**, writes it once, and the card replays it: nothing further crosses the FIFO until you change a setting (the status bar shows `GR_updates=0/s`). A loop is seamless only when the tone fits it a whole number of times. With **Loop-coherent** ticked (the default), the generator therefore sends the nearest frequency that does. The line under *Tone* always shows what is really sent:

```
Tone 10.000000 MHz
Transmits 9.999847 MHz – the nearest seamless frequency
(1 MiB H2C buffer, 262144-sample loop, step 762.939 Hz).
```

With the 16 KiB buffer of 1.2.1 the step was 48.8 kHz. With 16 MiB (forced through `IWFG_H2C_CHUNK_BYTES`) it is 47.7 Hz. Unticking *Loop-coherent* streams the exact frequency as a continuous series of chunks, which is what GNU Radio did. `iwfg_h2c` then has to copy every new chunk in between transfers, so the output is less clean. This is not recommended.

Stop sends a silent chunk before closing, so the card is not left replaying the last tone.

Using your own GNU Radio flowgraph instead

Leave the built-in generator **stopped** (two writers on one FIFO corrupt each other). Set *DAC input source* to *Host / GNU Radio stream*, then run your flowgraph into `/tmp/iwfg_h2c.fifo` as before. `iwfg_h2c` publishes a new chunk only once a full buffer (1 MiB by default) has arrived, and replays the last one meanwhile. A continuous GNU Radio stream is therefore cut into loops, with a short copy between transfers each time a new one arrives. For a clean tone from GNU Radio, send a signal that repeats exactly every buffer: a frequency that is a multiple of `fs / (buffer bytes / 4)`.

3. Simulation

In Simulation mode (no hardware) the plots can show what the transmit settings would produce through a DAC → ADC loopback:

DAC input	Generator	Plots show
DDS compiler	any	the DDS tone at the DDS frequency
Host / GNU Radio	running	the generator's exact waveform
Host / GNU Radio	stopped	noise only, as on the board

- *DAC input source* and *DDS frequency* on the RFDC tab stay usable with no device. **Start generator** runs a model; nothing is written to the FIFO.
- The **Simulation** box on the RFDC tab switches the loopback on and off. It turns on by itself the first time you touch a transmit control. Untick it to go back to the modem demo signal.
- The status bar names what is shown. It also says when a tone lies outside the displayed band instead of letting it alias into view.
- The loopback assumes matched NCOs (DAC NCO = ADC NCO, as at boot), so the transmitted baseband frequency is the received one.

4. Checking it without the board

```
TS/01_Host_Test_Suite/run_all.sh                                # everything below, one command
TS/01_Host_Test_Suite/firmware/run_tests.sh                    # firmware routing, 17 checks
cmake -S TS/01_Host_Test_Suite/pcie_emu -B /tmp/emu && cmake --build /tmp/emu && ctest --test-dir /tmp/emu
cmake -S TS/01_Host_Test_Suite/gui -B /tmp/gt && cmake --build /tmp/gt && QT_QPA_PLATFORM=offscreen ctest --test-dir /tmp/gt
cmake -S TS/01_Host_Test_Suite/h2c_emu -B /tmp/h2c && cmake --build /tmp/h2c && ctest --test-dir /tmp/h2c
```

`h2c_emu` runs the real `iwfg_h2c` and `c2h_stream` with the card replaced by `fake_iwfg.so`. The fake loops what the "DAC" plays back into the "ADC". It drives them from the real launcher, generator and reader, and also runs the whole application in a live session. It injects the bench faults:

- a C2H driver error;
- a card that refuses large transfers;
- a card that never consumes, or starts consuming late;

- helpers that ignore SIGINT and SIGTERM.

It checks that the generated tone comes back and that the GUI never blocks.

`pcie_emu` runs the GUI's real PCIe code against the firmware's real `pcie_cfg.c` through an emulated BAR. It does this for both 1.2.1 and 1.2.0 firmware, and after every command it checks what the DAC would actually play.

Capture and transmit together

Why the capture rate matters to transmit

When the FPGA's C2H FIFO overflows, the `iwfg` driver recovers by soft-resetting the whole QDMA and rebuilding every queue. The H2C queue is rebuilt too, even while a transfer is waiting on it. Each recovery stops the DAC feed for at least 17 ms. The DAC input has no flow control, so during that time it plays stale data. A capture that asks for more than the host can drain therefore keeps interrupting transmit, and it can wedge the device.

Capture	Need	Fits the 1600 MiB/s transmit budget?
4 ch × 200 MSPS	3052 MiB/s	No: overflows on the bench host (~2400 MiB/s drain)
2 ch × 200 MSPS	1526 MiB/s	Yes
1 ch × 200 MSPS	763 MiB/s	Yes

What the GUI does (1.2.5)

Mechanism	Behaviour
Transmit guard	When the generator starts, the GUI reduces a capture above 1600 MiB/s to 2 channels (or 1) through the PL GPIO mode. The RFDC tab must be attached for this. The GUI then checks the result against the measured rate. When the generator stops, it restores the original setting.
Ordered stop	Transmit is closed first, then capture. The capture's close is the last one, so the driver's reset runs and every new session starts clean.
Stalled-capture device reset	If the capture delivers nothing for about 5 s, the GUI stops transmit, then capture, then starts both again. It does this at most twice in 3 minutes. After that it asks you to reload the driver.
Same-size reopen	A transmit helper whose transfers stop completing is reopened at the same buffer size before any step-down.
Chunk step-down	1 MiB → 256 KiB → 64 KiB → 16 KiB, if the card refuses large transfers. The GUI returns to 1 MiB once the card consumes again.
Capture helper restart	A <code>c2h_stream</code> that dies mid-stream is restarted, and the display re-attaches. This happens up to 3 times a minute, without a dialog.
Larger capture pipe	The FIFO pipe is raised to 1 MiB (it was 64 KiB), up to <code>fs.pipe-max-size</code> .

Route your looped-back ADC to **AXIS output 0** so that it stays on screen when the guard reduces the channel count.

Planning another configuration

Open the *Calculations* sheet of the API workbook and change the inputs, for example the channels, the decimation, the chunk size or your host's measured drain. It recomputes the capture need, the guard decision, the PL GPIO mode, the H2C efficiency and the tone resolution.

Firmware serial console

The firmware's UART menu (PS UART0, 115200 8N1) offers the same functions as the PCIe control path. PCIe commands are still served while the menu waits for input: `MenuReadLine` polls `PcieCfg_Poll()` between keystrokes.

Menu	Function
1	ADC decimation, all enabled blocks. Re-runs ADC MTS and re-applies the ADC NCO.
2	ADC decimation, one tile/block
3	DAC interpolation, all enabled blocks. Re-runs DAC MTS and re-applies the DAC NCO.
4	DAC interpolation, one tile/block
5	Print the current decimation and interpolation rates
6	DDS sine frequency (MHz)
7	ADC NCO frequency, all channels (MTS-safe)
8	DAC NCO frequency, all channels (MTS-safe)

9	ADC AXIS switch routing: input 0-7 for each of the outputs 0-3
10	PL GPIO channel routing mode 1-3 (1, 2 or 4 channels)
11	Measure the stream clocks and the PL input clock
12	QMC gain, 0.0-1.9999, on all enabled ADC and DAC blocks
13	ADC or DAC sampling rate through the internal PLL (MHz). Saves and restores the QMC gain, then re-runs MTS and the NCO.
14	Force Nyquist zone 1 or 2 (ADC or DAC)
15	ADC DSA attenuation for one tile/block, 0.0-27.0 dB
16	DAC input source: 0 = DDS, 1 = Host/GNU Radio stream. Read back from the switch.
17	PCIe interface diagnostics
18	Set the PCIe control base address (debug), then run the diagnostics

Command-line control: `rfdc_ctl`

`rfdc_ctl` drives the same PCIe mailbox as the GUI. Global options: `-b/--bdf <BDF>` (sysfs backend), `-r/--bar <n>`, `-c/--chardev <path>`, `-o`, `-t`, `-v`, `-h`.

Command	Arguments	Function
<code>list</code>		List PCIe endpoints (opens no device)
<code>ping</code>		Handshake test that changes nothing
<code>dump</code>		Read and decode <code>ring_register</code> and <code>RFDC_configuration</code>
<code>raw-read</code>	<code><off_hex></code>	Read one 32-bit register
<code>raw-write</code>	<code><off_hex> <val_hex></code>	Write one 32-bit register
<code>nco-freq</code>	<code><adc dac both> <tile all> <ch both> <MHz></code>	NCO frequency
<code>nco-phase</code>	<code><adc dac both> <tile all> <ch both> <deg></code>	NCO phase, $-180..180$
<code>decim</code>	<code><tile all> <ch both> <factor></code>	ADC decimation
<code>interp</code>	<code><tile all> <ch both> <factor></code>	DAC interpolation
<code>dds-freq</code>	<code><MHz></code>	PL DDS frequency, $0 < f \leq 75$ MHz
<code>sample-rate</code>	<code><adc dac> <MHz></code>	Reprogram the PLL, re-align MTS, restore the NCO
<code>nyquist</code>	<code><adc dac> <1 2></code>	Force the Nyquist zone
<code>dsa</code>	<code><tile all> <ch both> <dB 0..27></code>	ADC attenuation
<code>qmc-gain</code>	<code><adc dac both> <tile all> <ch both> <0..1.9999></code>	QMC gain
<code>dac-vop</code>	<code><tile all> <ch both> <mA 2.25..40.5></code>	DAC output current
<code>axis-route</code>	<code><ch0> <ch1> <ch2> <ch3> (0..7 each)</code>	ADC AXIS routing
<code>gpio-route</code>	<code><1 2 3></code>	PL channel-select mode
<code>dac-source</code>	<code><dds host 0 1></code>	DAC input: DDS tone or H2C stream
<code>mts</code>		Re-align MTS and restore the NCOs
<code>start / stop</code>		Enable / disable all FIFOs
<code>reset</code>		Soft reset (MTS + NCO + routing)

Example: set both NCOs to 3100 MHz on all tiles, then confirm.

```
sudo ./rfdc_ctl nco-freq both all both 3100
sudo ./rfdc_ctl dump
```

Troubleshooting

Where to look first

Source	What it tells you
<code>\$TMPDIR/sdr_backend.log</code> (normally <code>/tmp/sdr_backend.log</code>)	Everything the helpers printed, and why a helper stopped. Send this file with any report.
GUI H2C TRANSMIT CHAIN rows	The state of each stage, read from the device where possible
<code>dmesg grep -i iwfg</code>	Driver messages, including "Failed to acquire reset lock ... forcing reset" on C2H overflow
UART console	Firmware ABI, init results, PCIe diagnostics (menu 17)

Symptoms and remedies

Symptom	Likely cause	Remedy

Capture shows 0.0 MB/s , or "C2H capture stopped"	C2H overflow recovery wedged the QDMA, or the helper died	The GUI resets the device by itself (at most twice in 3 min). If that keeps happening, reduce the channel count, or <code>rmmod iwfg && insmod iwfg.ko</code> .
Generator says "Reader disconnected (iwfg_h2c stopped?)"	Transmit helper stopped or was cycled	Read <code>sdr_backend.log</code> . Use Retry transmit . Check that <code>/dev/iwfg1</code> exists.
Transmitted tone has broadband junk	DAC starved (chunk too small) or capture overflowing	Keep the 1 MiB chunk. Let the transmit guard reduce the capture to 2 channels.
No tone at all with the generator running	DAC input is still on the DDS, or the RFDC tab is not attached	Attach, then set DAC input = Host , or use menu 16 → 1
ADC row warns QMC gain 0.0000	The QMC gain was zeroed	Set the QMC gain to 1.0 on the RFDC tab, or with menu 12
ENOB dropped (e.g. 11 → 6-7 bits)	Decimation reads 1× (boot value is 24×; this costs 13.8 dB, 2.3 bits), no tone, or the DAC is on an idle host stream	Power-cycle, or use menu 1 → 24, 3 → 24, 12 → 1.0, 16 → 0, then menu 5
"Firmware ABI 3, but this application needs ABI 4"	Partially flashed board	Reflash all firmware files together
GLIBC_2.38 not found	Prebuilt binary on an older distribution	Build from SC. Delete any old <code>backend/bin</code> .
H2C helper exits at once	<code>/dev/iwfg1</code> missing, or a stale helper binary	<code>ls -l /dev/iwfg*</code> ; rebuild the GUI (the helpers are rebuilt with it)
Peak at the band edge, 13-16 dB over the floor	There is no tone; SINAD is measuring noise	Check the DAC source, the cabling and the NCO frequencies

Getting back to a known state

- **Power-cycle the board.** This restores every boot default.
- **RFDC tab ► Reset all to defaults.** This sets 4800/9600 MSPS, ×24, 32 mA, routing 7/3/2/1 and the DDS.
- **Driver:** stop the GUI, then `sudo rmmod iwfg`, then `sudo insmod iwfg.ko`.

Known limitations

1. **The driver's C2H overflow recovery also resets H2C.** The GUI works around this (transmit guard, ordered stop, device reset), but the driver itself is unchanged in REL1.0. `UM/02_Developer_Notes/DRIVER_OVERFLOW_RECOVERY.md` lists the recommended driver fixes for the driver maintainer: keep TX out of the C2H recovery, guard the queue against use-after-free, re-apply `H2C_CTRL.VID_OUT_SEL`, replace `udelay(17000)` with `msleep()`, and report overflows to user space.
2. **A 4-channel capture at 200 MSPS exceeds the measured host drain.** It is usable for capture-only sessions, with overflows. It is not usable while transmitting.
3. **H2C buffers of 16 MiB do not complete** on the bench. The default is 1 MiB.
4. **The PL design files are not part of this package.** These are the XSA, bitstream and hwh. `HF_03_PL_Design` lists what must be supplied.
5. **The host-side tests use emulated hardware.** Bench acceptance is recorded with the ATP on the target board.

Appendix A: PCIe control mailbox (summary)

Register (BAR 2)	Offset	Use
<code>ring_register</code>	0xA8	Command word: <code>NEW_CMD[31]</code> <code>TARGET[30:28]</code> <code>LAST_TILE[27]</code> <code>TILE[26:24]</code> <code>CHANNEL[23:20]</code> <code>EVENT[19:0]</code>
<code>RFDC_configuration</code>	0xAC	Payload in, readback answer out

Payload	Encoding	Example
NCO frequency (event 1)	kHz, bit 31 = sign	4.5 GHz → <code>0x0044AA20</code>
NCO phase (event 2)	degrees 0..180, bit 31 = sign	−180 → <code>0x800000B4</code>
DDS frequency (event 5)	Hz	10 MHz → <code>0x00989680</code>
ADC DSA (0x12)	dB × 10, 0..270	6 dB → <code>60</code>
QMC gain (0x13)	gain × 10000	1.0 → <code>10000</code>
AXIS routing (0x14)	<code>ch0 ch1<<4 ch2<<8 ch3<<12</code>	default <code>0x1237</code>
ACK (written by the PS)	<code>0x2A<<24 err<<20 event</code>	OK on event 1 → <code>0x2A000001</code>
Readback request	<code>0x5242<<16 id</code>	QMC gain → <code>0x5242000D</code>

The workbook sheets *PCIe Control* (103 entries) and *Calculations* §2 give the complete map and an encoder for any command.